TITAN: A Hardware-Accelerated NTT Engine for Post-Quantum Cryptography on ARM64 NEON and AVX2

Andrea Catino
Independent Researcher, Italy
Via L. Marziani 13/d, Giovinazzo (BA), Italy
catino.andrea@gmail.com
https://github.com/catinoandrea-lgtm/TITAN-PQC

April 2026

Abstract

We present **TITAN**, an optimized ML-KEM-512 (FIPS 203) implementation featuring a high-performance NTT engine for ARM64 NEON and x86_64 AVX2. We evaluate two architectures: **TITAN 1** (open-source, hybrid NEON/scalar NTT) and **TITAN 3** (proprietary Zero-NTT architecture eliminating intermediate inverse transforms during CCA-secure decapsulation). On identical hardware, TITAN 1's scalar path is **1.11× faster** than the *pqcrystals/kyber* reference (121.64 μs vs. 135.31 μs). With NEON, TITAN 1 achieves **31.60 μs** on Apple M1 (**4.28**× vs. reference). TITAN 3's Zero-NTT architecture further reduces latency by **22–28%**, achieving **18.00 μs** on Apple M1 and **43.33 μs** on Samsung S23 Ultra (Snapdragon 8 Gen 2). A 9-test KAT suite with deterministic cross-platform fingerprint verifies correctness.

1 Introduction

The standardization of ML-KEM (FIPS 203) [1] has created an urgent need for efficient implementations on mobile and IoT devices. The NTT is the computational bottleneck: for ML-KEM-512 over $R_q = \mathbb{Z}_q[X]/(X^{256}+1)$ with $q = 3329$, optimizing the NTT directly translates to faster key exchange.

We present TITAN, a multi-architecture NTT engine with two implementation tiers:

TITAN 1 (Open Source): A hybrid NEON/scalar NTT with 16-bit Montgomery multiplication, Barrett reduction at every butterfly, streaming SHAKE-128 matrix generation, and FIPS 203 conformant KEM. Measured 1.11× faster than the *pqcrystals* reference on scalar C; 4.28× with NEON on Apple M1.

TITAN 3 (Proprietary, Zero-NTT Architecture): A re-engineered dataflow that eliminates intermediate INTT conversions during the CCA-secure protocol cycle. By maintaining all polynomial operations in a continuous spectral domain ("spectral tunnel"), TITAN 3 achieves 22–28% additional speedup over TITAN 1.

2 Background

2.1 ML-KEM (FIPS 203)

ML-KEM-512 [1] uses module rank $k = 2$, modulus $q = 3329$, CBD($\eta = 2$). Public key: 800 B, secret key: 1632 B, ciphertext: 768 B, shared secret: 32 B. KeyGen accepts 64 bytes ($d \| z$).

2.2 Number Theoretic Transform

The NTT operates modulo $q = 3329$ ($q \equiv 1 \pmod{256}$) with primitive root $\zeta = 17$. Cooley–Tukey [4] forward and Gentleman–Sande [5] inverse butterflies. Twiddle factors pre-stored in Montgomery form.

2.3 Montgomery Multiplication

For $R = 2^{16}$, $Q = 3329$: $Q_{\text{INV}} = -3327 \pmod{2^{16}}$, $R^2 \bmod Q = 1353$, $N^{-1} \cdot R \bmod Q = 512$. Barrett reduction $v = 20159 \approx 2^{26}/Q$ [3].

3 TITAN 1 Architecture (Open Source)

3.1 Hybrid NEON/Scalar NTT

The forward NTT uses NEON 8-wide SIMD for stride ≥ 8 (5 levels) with Barrett after each level, and scalar with Barrett at every butterfly for stride 4 and 2. The `fqmul_neon` core computes 8 modular multiplications in 6 instructions: `vmull_s16` (2×), `vmovn_s32` + `vmulq_s16`, `vmlsl_s16` (2×), `vshrn_n_s32` (2×).

3.2 Montgomery Domain Tracking

All arithmetic operates in Montgomery domain with explicit `tomont`/`frommont` gates at boundaries. NTT zetas are pre-stored in Montgomery form.

3.3 Streaming Matrix Generation

Incremental SHAKE-128 squeeze (168 B/block) with guaranteed termination.

3.4 Constant-Time Decapsulation

Bitwise OR accumulation and arithmetic masking for implicit rejection.

4 TITAN 3: Zero-NTT Architecture (Proprietary)

4.1 Motivation

Standard ML-KEM implementations perform repeated NTT↔time-domain conversions. Each INTT/NTT pair introduces overhead and flushes the vector pipeline.

4.2 Spectral Tunnel Design

TITAN 3 exploits NTT linearity: $\text{NTT}(a+b) = \text{NTT}(a)+\text{NTT}(b)$. Error additions are performed directly in NTT domain (error pre-transformed). The protocol maintains all intermediates in NTT domain:

Standard: NTT → Multiply → INTT → Add error → NTT → Re-encrypt → INTT
TITAN 3: NTT → Multiply → Add (NTT domain) → CCA verify → INTT (final only)

This eliminates 2–4 intermediate inverse transforms per protocol operation.

4.3 Hardware Optimizations

32-byte aligned memory for AVX2, `restrict` pointers for full unrolling, interleaved NEON load/store (`vld2q_s16`/`vst2q_s16`).

Note: TITAN 3 source code is proprietary. This section describes the architecture; the implementation is available under commercial license.

5 Implementation

TITAN 1: C99, ∼900 lines ARM NEON KEM + ∼160 lines AVX2 math engine, MIT license. TITAN 3: C11, compile-time version selection, multi-architecture dispatch (NEON/AVX2/scalar). Shared Keccak-f[1600], SHA-3, SHAKE, CBD, matrix generation.

6 Experimental Results

6.1 Platforms

Table 1: Evaluation platforms.

Platform	**CPU**	**Arch**	**Freq**	**Compiler**	**SIMD**
Apple M1 (CI)	Firestorm	ARM64	3.2 GHz	Clang 15	NEON
Samsung S23 Ultra	Cortex-X3	ARM64	3.36 GHz	Clang 9	NEON
Android mid-range	A55/A78	ARM64	2.0 GHz	Clang 9	NEON
GitHub CI x86	Xeon/EPYC	x86_64	∼2.8 GHz	GCC 13	(scalar)
AMD EPYC 7742	Zen 2	x86_64	2.25 GHz	GCC 11	AVX2

6.2 TITAN 1: Baseline Comparison (Scalar)

Table 2: TITAN 1 vs. pqcrystals reference (scalar, same hardware).

Operation	**TITAN 1 (μs)**	**Ref C (μs)**	**Ratio**
Keygen	30.23	35.76	**1.18**×
Encaps	44.78	42.49	0.95×
Decaps	46.62	57.07	**1.22**×
Total	**121.64**	**135.31**	**1.11**×

6.3 TITAN 1: NEON Performance

Table 3: TITAN 1 with NEON. Speedup vs. ref C (135.31 μs).

Platform	**KG**	**Enc**	**Dec**	**Total**	**vs. Ref**
Apple M1	8.13	11.55	11.92	**31.60**	**4.28**×
Samsung S23 Ultra	26.16	34.04	28.60	**88.79**	**1.52**×
Android mid-range	63.87	84.88	94.68	243.43	0.56×

Table 4: TITAN 3 vs. TITAN 1. Median latency (μs).

Platform	**TITAN 1**	**TITAN 3**	**Impr.**	**vs. Ref**
Apple M1 (NEON)	25.00	**18.00**	−28.0%	**7.52**×
AMD EPYC 7742 (AVX2)	83.50	**63.22**	−24.2%	2.14×
Intel Xeon (AVX2)	112.55	**87.42**	−22.3%	1.55×
Samsung S23 Ultra	57.03	**43.33**	−24.0%	**3.12**×

6.4 TITAN 3: Zero-NTT Performance

6.5 NTT Engine

Table 5: Isolated NTT benchmarks (μs).

Platform	**Fwd**	**Inv**	**PW**	**Kx4**
Apple M1 (NEON)	**0.29**	**0.25**	—	—
Samsung S23 Ultra (NEON)	0.87	0.51	—	—
Android mid-range (NEON)	4.15	1.88	—	—
AMD EPYC 7742 (AVX2)	2.40	1.32	0.020	1.36
Google Colab (AVX2)	6.08	1.92	0.016	1.70

6.6 KAT Conformance

TITAN 1 KAT suite: 9 tests (SHA3, SHAKE, NTT roundtrip, sizes, determinism, correctness, implicit rejection, 100 accumulated vectors). Cross-platform fingerprint:

`807c873a3ad4a624b16bfcd44018d8cf0949da547561fe99daf5039cdf1342bd`

Verified on x86_64 Ubuntu, GitHub CI, Samsung S23 Ultra, Android mid-range.

6.7 Analysis

Three findings: (1) TITAN 1 scalar outperforms pqcrystals by 11%, proving gains from streaming SHAKE-128, Barrett placement, and domain tracking beyond SIMD. (2) ARM NEON NTT (0.29 μs on M1) is 8.3× faster than AVX2 on EPYC (2.40 μs). (3) TITAN 3 provides consistent 22–28% reduction across all platforms, confirming INTT elimination is a genuine architectural improvement.

7 Related Work

The pqcrystals/kyber reference [2] provides portable C and AVX2. Our baseline (Table 2) shows TITAN 1 is 1.11× faster. The mlkem-native project [6] targets multi-platform ML-KEM. Becker et al. [8] demonstrated NEON NTT. Stelzer et al. [7] investigated OpenTitan extensions. TITAN differentiates through its dual open/proprietary model, mobile validation, and Zero-NTT spectral tunnel.

8 Future Work

NIST ACVP validation, ML-KEM-768/1024 extension, SVE2 support, telescoping series compression, Zero-NTT for ML-DSA (FIPS 204).

9 Conclusion

We presented TITAN, a two-tier ML-KEM-512 implementation. TITAN 1 (open source) achieves **4.28× speedup** over pqcrystals on M1. TITAN 3 (proprietary Zero-NTT) reduces latency by 22–28%, achieving **18 μs** on M1 and **43 μs** on Samsung S23 Ultra. These results demonstrate that post-quantum key exchange at sub-100 μs is achievable on commodity mobile hardware, and that architectural dataflow innovations—not just SIMD—unlock the next performance level.

References

[1] NIST, "ML-KEM Standard," FIPS 203, Aug. 2024.

[2] R. Avanzi et al., "CRYSTALS-Kyber," NIST PQC Round 3, 2021.

[3] P. L. Montgomery, "Modular Multiplication Without Trial Division," *Math. Comp.*, 44(170), 1985.

[4] J. W. Cooley, J. W. Tukey, "Machine Calculation of Complex Fourier Series," *Math. Comp.*, 19(90), 1965.

[5] W. M. Gentleman, G. Sande, "FFT: For Fun and Profit," FJCC, 1966.

[6] pq-code-package, "mlkem-native," GitHub, 2024.

[7] F. Stelzer et al., "Lattice-Based PQC on OpenTitan," ASHES, 2023.

[8] H. Becker et al., "Neon NTT," TCHES, 2022.

[9] ARM Ltd., "ARMv8-A Reference Manual," 2024.

[10] NIST, "SHA-3 Standard," FIPS 202, 2015.

A Availability

TITAN 1: MIT License, https://github.com/catinoandrea-lgtm/TITAN-PQC.
TITAN 3: Architecture in Section 4. Source under commercial license.